音视频同步功能实现方案

一、需求总结

项目	说明
开发目标	将音视频同步逻辑从 ckernel 中抽离，重写为独立的 AVSyncManager 模块
同步策略	音频主时钟（audio_clock 为基准，视频同步到音频）
同步范围	接收端（远端音视频同步显示）+ 发送端（时间戳对齐 + 本地预览同步显示）
多用户支持	每个远端用户独立维护一套 audio_clock / video_clock
文件位置	现有模块目录内（VideoApi 目录下）
代码规范	严格遵守：不删除原有代码（注释保留）、所有变更添加注释说明

二、架构设计

2.1 模块结构

```
WeChat/VideoApi/
├── AVSyncManager.h      // 新增：音视频同步管理器头文件
├── AVSyncManager.cpp    // 新增：音视频同步管理器实现
├── videoread.h/cpp      // 现有：视频采集（需集成发送端同步）
└── ...
```

2.2 核心类设计

```
AVSyncManager（单例）
│
├── SyncContext 结构体（每用户独立）
│   ├── audioClock: double        // 音频主时钟（微秒）
│   ├── videoClock: double        // 视频辅助时钟（微秒）
│   ├── lastAudioUpdateTime: qint64  // 上次音频时钟更新时间戳
│   ├── lastVideoUpdateTime: qint64  // 上次视频时钟更新时间戳
│   └── isPaused: bool            // 该用户是否暂停
│
├── 公共接口
│   ├── getInstance()              // 获取单例
```

```
│   ├── addUser(userId)           // 添加用户同步上下文
│   ├── removeUser(userId)        // 移除用户同步上下文
│   ├── updateAudioClock(userId, pts)  // 更新音频时钟
│   ├── synchronizeVideo(userId, pts, frameDelay) -> int  // 视频同步等待，返回需等待毫秒数
│   ├── getAudioClock(userId)     // 获取音频时钟
│   ├── getVideoClock(userId)     // 获取视频时钟
│   ├── resetUserClock(userId)    // 重置用户时钟
│   ├── resetAllClocks()          // 重置所有时钟
│   ├── pauseUser(userId)         // 暂停用户同步
│   └── resumeUser(userId)        // 恢复用户同步
│
├── 同步参数（可配置）
│   ├── SYNC_THRESHOLD_US: 10000   // 同步阈值 10ms（微秒）
│   ├── MAX_WAIT_MS: 100           // 最大等待时间 100ms（毫秒）
│   └── CLOCK_TIMEOUT_MS: 5000     // 时钟超时 5s（毫秒）
│
└── 线程安全
    └── std::mutex m_mutex         // 保护 SyncContext map 的访问
```

2.3 同步算法（音频主时钟策略）

```
synchronizeVideo(userId, video_pts, frameDelay):
    1. 获取该用户的 SyncContext
    2. 更新 videoClock = video_pts（若 PTS > 0）
    3. 计算当前帧的显示 PTS = videoClock
    4. 推进 videoClock += frameDelay（为下一帧准备）
    5. 比对：diff = video_pts - audioClock
       - diff > SYNC_THRESHOLD → 视频超前，等待 diff 毫秒（最大 MAX_WAIT_MS）
       - -SYNC_THRESHOLD <= diff <= SYNC_THRESHOLD → 同步正常，立即显示
       - diff < -SYNC_THRESHOLD → 视频落后，立即显示（丢弃策略可选）
    6. 返回需等待的毫秒数（0 = 立即显示）
```

三、集成方案

3.1 接收端集成（ckernel.cpp）

音频帧接收路径 `slot_dealAudioFrameRq`：

- 现有：用 QTime::currentTime() 更新 m_audioClock
- 修改为：调用 AVSyncManager::updateAudioClock(userId, timestamp)
- 时间戳来源：从音频包头中的 min/sec/msec 字段计算

视频帧接收路径（JPEG） `slot_dealVideoFrameRq`：

- 现有：直接调用 synchronizeVideo(video_pts, frameDelay)
- 修改为：调用 AVSyncManager::synchronizeVideo(userId, video_pts, frameDelay)

- 根据返回值决定是否 msleep

视频帧接收路径（H.264） `slot_dealVideoFrameH264Rq`：

- 同上，增加 userId 参数调用 AVSyncManager

3.2 发送端集成（ckernel.cpp）

时间戳对齐：

- 发送音频帧时：从 AVSyncManager 获取当前时间戳，写入包头
- 发送视频帧时：从 AVSyncManager 获取同步后的时间戳，写入包头
- 确保同一时刻采集的音视频帧携带一致的时间基准

本地预览同步显示 `slot_sendVideoFrame`：

- 在本地显示视频前，调用 AVSyncManager 进行同步等待
- 本地音频采集（AudioRead）的时钟作为主时钟基准

3.3 生命周期管理

- 创建房间/加入房间时：初始化 AVSyncManager，添加本地用户
- 收到新成员时：调用 addUser(userId)
- 成员离开时：调用 removeUser(userId)
- 退出房间时：调用 resetAllClocks()
- 暂停/恢复音频/视频时：调用 pauseUser/resumeUser

四、文件变更清单

文件	变更类型	说明
WeChat/VideoApi/AVSyncManager.h	新增	同步管理器头文件
WeChat/VideoApi/AVSyncManager.cpp	新增	同步管理器实现
WeChat/VideoApi/videoapi.pri	修改	添加 AVSyncManager 到构建
WeChat/ckernel.h	修改	添加 AVSyncManager 成员，保留原同步成员（注释化）
WeChat/ckernel.cpp	修改	集成 AVSyncManager 调用，保留原同步代码（注释化）

文件	变更类型	说明
WeChat/vcc-client.pro	修改（如需）	添加新文件到构建

五、关键设计决策

1. **单例模式**：AVSyncManager 使用单例，便于 ckernel 和其他模块统一访问
2. **每用户独立上下文**：使用 std::map<int, SyncContext> 管理多用户时钟
3. **互斥锁保护**：所有时钟操作通过 std::mutex 保护线程安全
4. **返回等待时间而非直接sleep**：synchronizeVideo 返回需等待毫秒数，由调用方决定是否sleep，保持模块职责单一
5. **时间戳统一使用微秒**：内部统一使用微秒（double），避免精度丢失

六、待确认事项

无（所有关键问题已在前期沟通中确认）。